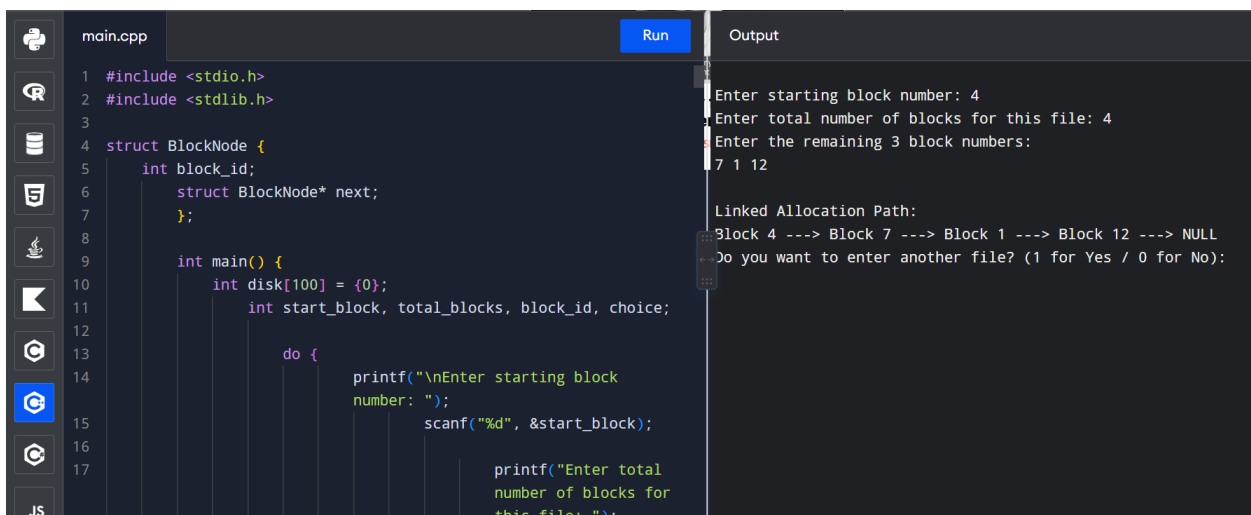


EXPERIMENT 30 :

30. With linked allocation, each file is a linked list of disk blocks; the disk blocks may be scattered anywhere on the disk. The directory contains a pointer to the first and last blocks of the file. Each block contains a pointer to the next block. Design a C program to simulate the file allocation strategy.

AIM:

To design a C program to simulate the Linked File Allocation strategy.

The image shows a code editor with a C program named 'main.cpp' and its output. The code defines a 'BlockNode' struct with 'block_id' and 'next' pointer, and a 'main' function that simulates the linked allocation process. The output shows the user inputting starting block 4, total blocks 4, and remaining blocks 7, 1, 12, resulting in a linked path: Block 4 ---> Block 7 ---> Block 1 ---> Block 12 ---> NULL.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct BlockNode {
5     int block_id;
6     struct BlockNode* next;
7 };
8
9 int main() {
10     int disk[100] = {0};
11     int start_block, total_blocks, block_id, choice;
12
13     do {
14         printf("\nEnter starting block
15         number: ");
16         scanf("%d", &start_block);
17
18         printf("Enter total
19         number of blocks for
20         this file: ");
21         scanf("%d", &total_blocks);
22
23         printf("Enter the remaining %d block numbers:", total_blocks - 1);
24         for (int i = 0; i < total_blocks - 1; i++) {
25             scanf("%d", &block_id);
26             disk[block_id] = 1;
27             if (i < total_blocks - 2) printf(" ");
28             else printf("\n");
29         }
30
31         printf("\nLinked Allocation Path:\n");
32         int current_block = start_block;
33         while (current_block != 0) {
34             printf("Block %d ---> ", current_block);
35             current_block = disk[current_block];
36         }
37         printf("NULL\n");
38         printf("Do you want to enter another file? (1 for Yes / 0 for No): ");
39         scanf("%d", &choice);
40     } while (choice == 1);
41 }
```

Output:

```
Enter starting block number: 4
Enter total number of blocks for this file: 4
Enter the remaining 3 block numbers:
7 1 12

Linked Allocation Path:
Block 4 ---> Block 7 ---> Block 1 ---> Block 12 ---> NULL
Do you want to enter another file? (1 for Yes / 0 for No):
```

PSEUDO CODE :

BEGIN

DECLARE STRUCT Node containing:

integer block_id

pointer next

END STRUCT

DECLARE disk array initialized to 0

LOOP

PRINT "Enter file starting block and total blocks needed:"

READ start_block, total_blocks

```
IF disk[start_block] == 1 THEN
```

```
    PRINT "Start block already occupied!"
```

```
    CONTINUE
```

```
END IF
```

```
DECLARE allocated_list array of size total_blocks
```

```
SET allocated_list[0] = start_block
```

```
SET valid = 1
```

```
PRINT "Enter remaining " + (total_blocks - 1) + " block IDs:"
```

```
FOR i FROM 1 TO total_blocks - 1 DO
```

```
    READ block_id
```

```
    SET allocated_list[i] = block_id
```

```
    IF disk[block_id] == 1 THEN
```

```
        SET valid = 0
```

```
    END IF
```

```
END FOR
```

```
IF valid == 0 THEN
```

```
    PRINT "Allocation Failed: One of the requested blocks is already  
occupied."
```

```
ELSE
```

```
    FOR i FROM 0 TO total_blocks - 1 DO
```

```
        SET disk[allocated_list[i]] = 1
```

```
    END FOR
```

```
PRINT "Linked List Representation:"
```

```
FOR i FROM 0 TO total_blocks - 2 DO
```

```
    PRINT "Block " + allocated_list[i] + " ---> "
```

```

        END FOR
        PRINT "Block " + allocated_list[total_blocks - 1] + " ---> NULL"
    END IF

    PRINT "Do you want to enter another file? (1-Yes, 0-No):"
    READ choice
    UNTIL choice == 0
END

```

CODE:

```

#include <stdio.h>

#include <stdlib.h>

struct BlockNode {

    int block_id;

    struct BlockNode*
next;

};

int main() {

    int disk[100] = {0};

    int start_block,

```

```
total_blocks, block_id,  
choice;
```

```
do {
```

```
    printf("\nEnter  
starting block number:  
");
```

```
    scanf("%d",  
&start_block);
```

```
    printf("Enter total  
number of blocks for  
this file: ");
```

```
    scanf("%d",  
&total_blocks);
```

```
    if  
(disk[start_block] == 1)  
{
```

```
        printf("Error:
```

```
Start block %d is
already occupied!\n",
start_block);

    } else {

        int
blocks_array[50];

        blocks_array[0]
= start_block;

        int conflict = 0;

        printf("Enter the
remaining %d block
numbers:\n",
total_blocks - 1);

        for (int i = 1; i <
total_blocks; i++) {

            scanf("%d",
&blocks_array[i]);

            if
(disk[blocks_array[i]]
== 1) {
```

```
        conflict = 1;
```

```
    }
```

```
}
```

```
if (conflict) {
```

```
    printf("Allocation
```

```
Failed: One or more
```

```
blocks are already in
```

```
use.\n");
```

```
    } else {
```

```
        struct
```

```
BlockNode* head =
```

```
NULL;
```

```
        struct
```

```
BlockNode* temp =
```

```
NULL;
```

```
        for (int i = 0; i
```

```
< total_blocks; i++) {
```

```
disk[blocks_array[i]] =  
1;
```

```
        struct  
BlockNode* newNode  
= (struct  
BlockNode*)malloc(siz  
eof(struct BlockNode));
```

```
        newNode->  
block_id =  
blocks_array[i];
```

```
        newNode->  
next = NULL;
```

```
        if (head ==  
NULL) {
```

```
            head =  
newNode;
```

```
            temp =  
head;
```

```
        } else {
```

```
        temp->next = newNode;

        temp =
temp->next;

    }

}
```

```
printf("\nLinked
Allocation Path:\n");
```

```
        temp = head;

        while (temp
!= NULL) {
```

```
printf("Block %d",
temp->block_id);
```

```
        if (temp->next != NULL)
```

```
            printf(" --
-> ");
```



```
        else

            printf(" --
-> NULL\n");

            temp =
temp->next;

        }

    }

}
```

```
    printf("Do you
want to enter another
file? (1 for Yes / 0 for
No): ");

    scanf("%d",
&choice);

    } while (choice == 1);

    return 0;

}
```